

SMART RESUME & JOB MATCHING SYSTEM

Project Report

Course: Python Essentials

Submitted By

Tanisi Agarwal

Registration Number: 26BCE10942

Course: Bachelor of Technology — Computer Science & Engineering

University: VIT Bhopal University

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Introduction 3
2. Problem Statement 3
3. Objectives 3
4. Scope of the Project 4
5. Target Users 4
6. Functional Requirements 4
7. Non-Functional Requirements 5
8. System Design and Methodology 7
9. Implementation Details 12
10. Testing and Results 26
11. Challenges Faced 31
12. Learning Outcomes 31
13. Limitations 31
14. Future Enhancements 31
15. Conclusion 32
16. References 32

1. Introduction

The rapid growth of online recruitment platforms has increased the number of job opportunities available to candidates. However, job seekers often need to evaluate their resumes against different job descriptions to determine whether their skills, education, and experience satisfy the requirements of a particular position. Manually performing this comparison can be time-consuming, especially when a job description contains a large number of technical and professional requirements.

The Smart Resume & Job Matching System is a command-line based application developed to provide an automated analytical approach to resume and job-description comparison. The system processes a candidate's resume in PDF format and compares its contents with a given job description.

The application extracts textual information from the resume using PyMuPDF, identifies technical skills using a configurable skills database, and compares the detected resume skills with the skills mentioned in the job description. It also uses TF-IDF vectorization and cosine similarity to measure textual similarity between the resume and job description.

In addition to skill matching, the system analyzes important resume sections and checks education- and experience-related requirements. The results are combined to generate a structured matching report containing matched skills, missing skills, similarity information, experience analysis, an overall matching percentage, and recommendations.

The system follows a modular software architecture in which different responsibilities are separated into individual Python modules. This improves maintainability, readability, testing, and future extensibility.

---

2. Problem Statement

Recruitment processes require candidates and recruiters to compare resumes with job descriptions containing technical skills, educational qualifications, experience requirements, and other relevant information.

A manual comparison requires the user to read both documents and identify:

- Skills already present in the resume
- Skills required by the job
- Missing technical skills
- Relevant education information
- Relevant experience information
- Similarity between the resume and job description

Performing these tasks manually can consume significant time and may result in inconsistent analysis.

Therefore, there is a need for a structured software solution that can automatically extract resume information, identify relevant skills, compare the resume with a job description, analyze important requirements, and present the results in an understandable format.

The Smart Resume & Job Matching System addresses this problem by providing a command-line based automated analysis system.

---

3. Objectives

1. To extract and process resume information from PDF files.
2. To identify technical skills in resumes and job descriptions.
3. To compare candidate skills with job requirements.
4. To calculate resume–job similarity using TF-IDF and cosine similarity.
5. To analyze education and experience requirements.
6. To generate a matching score and recommendations.
7. To provide the analysis through a simple command-line report.

---

4. Scope of the Project

The Smart Resume & Job Matching System analyzes a resume against a job description in a local command-line environment.

It supports:

- PDF resume and text-based job description input
- Skill detection and matching
- Missing skill identification
- TF-IDF and cosine similarity
- Education and experience analysis
- Overall match score and recommendations
- Command-line reporting
- Input validation and error handling

The current system uses sample input files and does not include a web interface, authentication, cloud deployment, or database management.

---

5. Target Users

5.1 Job Seekers

Job seekers can use the system to compare their resumes with specific job descriptions and identify technical skills that are present or missing.

5.2 Students

Students can use the application to understand how their current technical skills and project experience compare with requirements commonly mentioned in job descriptions.

5.3 Recruiters

Recruiters can potentially use the system as an initial analytical tool for examining resume content against specified job requirements.

5.4 Career Advisors

Career advisors can use the generated analysis to help candidates identify areas where their resumes may require additional skills or relevant information

---

6. Functional Requirements

Functional requirements define the main operations performed by the Smart Resume & Job Matching System.

6.1 Resume PDF Processing

The system shall accept a PDF resume and extract its text for analysis.

6.2 Job Description Processing

The system shall read and validate the job description from a text file.

6.3 Skill Processing

The system shall load skills from data/skills.txt and identify relevant skills in both the resume and job description.

6.4 Skill Matching

The system shall compare resume skills with job-required skills and report matched skills, missing skills, and the skill matching percentage.

6.5 Text Similarity Analysis

The system shall calculate similarity between the resume and job description using TF-IDF and cosine similarity.

6.6 Resume Analysis

The system shall detect important resume sections such as Skills, Education, Experience, and Projects.

6.7 Education and Experience Analysis

The system shall compare education and experience requirements from the job description with information available in the resume.

6.8 Overall Match Calculation

The system shall calculate an overall match percentage using the implemented scoring formula:

Overall Match = (Skill Match × 0.50) + (Text Similarity × 0.30) + (Experience Match × 0.20)

The weights are design choices for this project.

6.9 Recommendations

The system shall provide recommendations based on skills missing from the resume.

6.10 Report Generation

The system shall display a consolidated command-line report containing matching results, missing skills, and recommendations.

6.11 Input Validation and Error Handling

The system shall validate input files and handle missing files, invalid paths, empty content, and runtime errors.

7. Non-Functional Requirements

Non-functional requirements describe the quality attributes and operational characteristics of the system.

7.1 Usability

The application should provide clear command-line output so that users can easily understand the matching results.

7.2 Performance

The system should process the provided resume and job description within a reasonable amount of time for normal-sized input documents.

7.3 Reliability

The system should handle invalid or missing input files without unexpectedly terminating with an unhandled application error.

7.4 Maintainability

The application should use separate modules for different responsibilities, such as parsing, skill detection, matching, analysis, reporting, and validation.

7.5 Extensibility

The skills database should be configurable so that additional technical skills can be added without modifying the core skill-detection logic.

7.6 Portability

The application should be capable of running on systems with a compatible Python 3 environment and the dependencies specified in requirements.txt.

7.7 Accuracy

The system should consistently apply its implemented skill-matching and text-similarity algorithms to the same input data.

7.8 Error Handling

The application should provide understandable error messages when required files are missing, invalid, or contain empty content.

8. System Design and Methodology

8.1 System Architecture

The Smart Resume & Job Matching System uses a modular Python architecture. The main application coordinates different modules to process the resume and job description.

The system accepts:

1. Candidate resume in PDF format.
2. Job description in text format.
3. Technical skills database.

The parser extracts resume text, the skills module detects technical skills, and the matcher compares skills and text similarity. The analyzer checks resume sections and requirements, while the reporter generates the final results and recommendations.

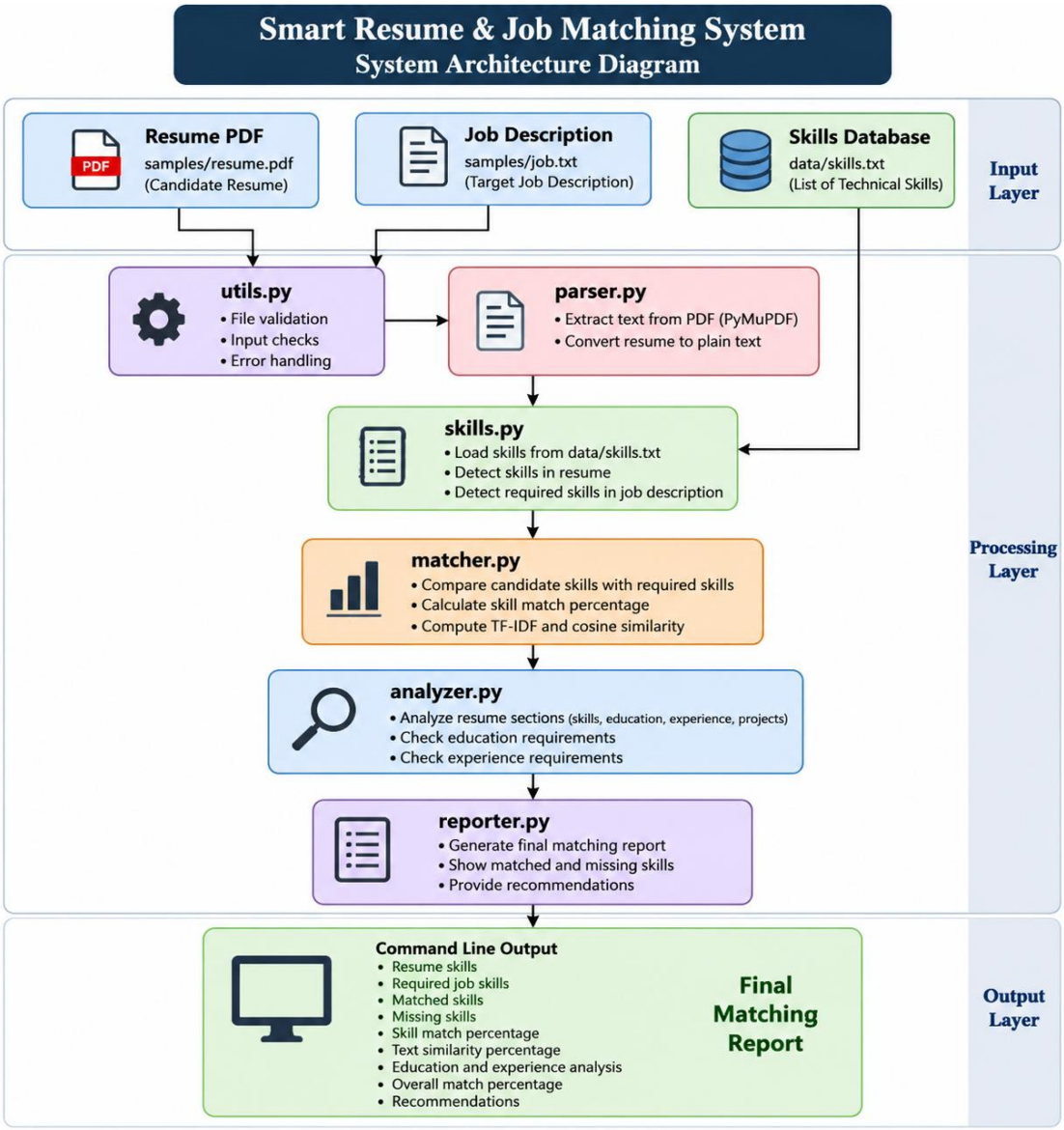


Figure 1: System Architecture of Smart Resume & Job Matching System

8.2 System Workflow

The system follows these main steps:

1. Start the application.
2. Validate input files.
3. Extract resume text from PDF.

4. Load job description and skills database.
5. Detect skills in the resume and job description.
6. Compare skills and calculate TF-IDF similarity.
7. Analyze resume sections, education, and experience.
8. Calculate the overall matching score.
9. Generate recommendations.
10. Display the final report.

Invalid or missing files are handled using error-handling mechanisms.

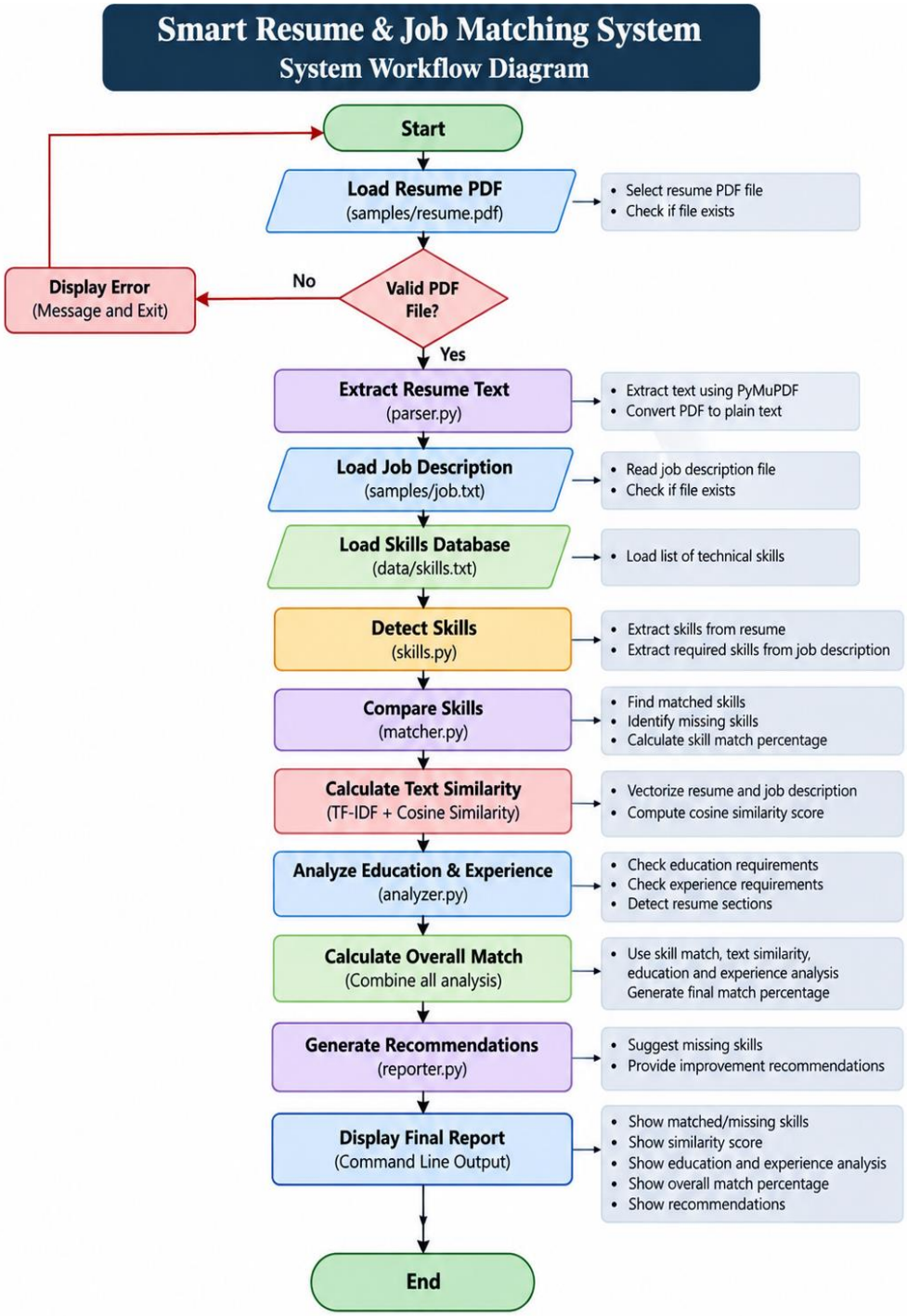


Figure 2: System Workflow of Smart Resume & Job Matching System

8.3 Use Case Design

The main users are job seekers, recruiters, and system administrators.

- **Job Seekers:** Upload resumes and job descriptions and view matching results.
- **Recruiters:** Analyze resumes according to job requirements.
- **Administrators:** Maintain the technical skills database.

The current system is command-line based, so these represent functional interactions rather than implemented login roles.

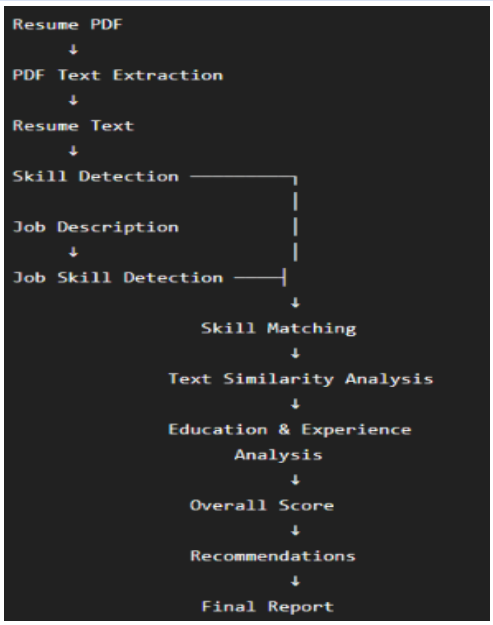
8.4 Component and Module Design

The application is divided into the following Python modules:

- **main.py:** Controls the complete application workflow.
- **parser.py:** Extracts text from PDF resumes using PyMuPDF.
- **skills.py:** Detects technical skills using the skills database and regular expressions.
- **matcher.py:** Performs skill matching and TF-IDF-based text similarity.
- **analyzer.py:** Analyzes resume sections, education, experience, and projects.
- **reporter.py:** Generates the final report and recommendations.
- **utils.py:** Provides file validation, text handling, and formatting functions.

8.5 Data Flow

Resume and job-description data are processed through the parser, skills detector, matcher, and analyzer modules. The skills database supports skill identification, while the reporter combines the results into the final matching report.



8.6 Matching Methodology

The system uses three main scoring components:

- Skill Match – 50%
- Text Similarity – 30%
- Experience Match – 20%

The overall score is calculated as:

Overall Match = (Skill Match × 0.50) + (Text Similarity × 0.30) + (Experience Match × 0.20)

These weights are project-specific and are not an industry-standard hiring formula.

8.7 Skill Matching

Skills are detected from both the resume and job description using the configured skills database.

The system identifies:

- Matched skills
- Missing skills
- Skill match percentage

Skill Match = (Matched Skills / Required Skills) × 100

8.8 Text Similarity Using TF-IDF

TF-IDF is used to represent the resume and job description as numerical vectors. It gives higher importance to terms that are significant within the documents.

The resulting vectors are compared using cosine similarity.

8.9 Cosine Similarity

Cosine similarity measures the similarity between two vectors:

Cosine Similarity = (A · B) / (||A|| × ||B||)

Here, **A** represents the resume vector and **B** represents the job-description vector. The result is converted into a percentage for the final report.

8.10 Recommendation Method

The system identifies missing skills and generates recommendations suggesting areas where the candidate can improve their knowledge.

8.11 Alignment with Python Essentials

Overall, the project applies Python fundamentals in an end-to-end workflow: input validation → file handling → text processing → data structures → conditional logic and loops → reusable functions/modules → algorithmic matching → exception handling → formatted output.

• Third-party Python libraries — PyMuPDF supports PDF text extraction, while Scikit-learn provides TF-IDF vectorization and cosine similarity. These libraries extend the Python fundamentals demonstrated by the project.

• String processing and regular expressions — used for text normalization, keyword detection, skill matching, and section analysis.

• Exception handling — used to handle missing files, invalid input, empty content, and unexpected runtime errors without uncontrolled termination.

• File handling — used to read the PDF resume, job-description text file, and configurable skills database.

• Lists and dictionaries — used to store skills, matched/missing skills, keywords, detected sections, and structured report results.

• Modules — functionality is separated across main.py, parser.py, skills.py, matcher.py, analyzer.py, reporter.py, and utils.py, demonstrating modular Python programming.

• Functions — the application is organized into reusable functions such as extract_pdf_text(), load_skills(), find_skills(), calculate_skill_match(), calculate_text_similarity(), analyze_education_requirement(), analyze_experience_requirement(), generate_report(), and display_report().

• Loops — used to traverse PDF pages, skills, resume lines, detected keywords, matched skills, missing skills, and report items.

• Conditional statements — used to validate files, detect matching or missing skills, evaluate resume sections, handle score conditions, and control report generation.

• Variables, expressions, and data types — used for paths, text values, percentages, counters, Boolean results, and intermediate analysis values.

The project demonstrates core Python Essentials concepts through a practical, modular application rather than using Python only as a supporting language.

9. Implementation Details

9.1 Implementation Overview

The Smart Resume & Job Matching System is implemented using Python and follows a modular design. Each major operation is separated into an independent Python module, while main.py coordinates the complete execution flow.

The implementation consists of modules for PDF parsing, skill detection, matching, resume analysis, report generation, and input validation.

The major implementation modules are:

- main.py
- parser.py
- skills.py
- matcher.py
- analyzer.py
- reporter.py
- utils.py

9.2 Main Application Module — main.py

The main.py file acts as the central controller of the application.

It coordinates the complete workflow by:

1. Establishing project file paths.
2. Validating input files.
3. Loading the resume.
4. Extracting resume text.
5. Loading the job description.
6. Loading the skills database.
7. Detecting resume and job skills.
8. Calculating skill matching.
9. Calculating text similarity.
10. Analyzing resume sections.
11. Analyzing education requirements.
12. Analyzing experience requirements.
13. Calculating the overall matching score.
14. Generating the final report.
15. Handling application errors.

The module uses functions imported from the other project modules rather than implementing all functionality in a single file.

This separation improves readability and maintainability.

```python
import os
from parser import extract_pdf_text
from skills import load_skills, find_skills
from matcher import calculate_skill_match, calculate_text_similarity
from analyzer import (
    detect_resume_sections,
    analyze_education_requirement,
    analyze_experience_requirement)
from reporter import generate_report, display_report
from utils import validate_file, read_text_file, validate_text
print("====================================")
print("     SMART RESUME & JOB MATCHER")
print("====================================")
print()
try:
    # ----------------------------------------
    # Project File Paths
    # ----------------------------------------
    project_folder = os.path.dirname(os.path.abspath(__file__))
    resume_file = os.path.join(project_folder, "samples", "resume.pdf")
    job_file = os.path.join(project_folder, "samples", "job.txt")
    skills_file = os.path.join(project_folder,"data", "skills.txt")
    # ----------------------------------------
    # Load Resume
    # ----------------------------------------
    validate_file(resume_file)
    resume_text = extract_pdf_text(resume_file)
    validate_text(resume_text, "Resume")
    # ----------------------------------------
    # Load Job Description
    # ----------------------------------------
    validate_file(job_file)
    job_text = read_text_file(job_file)
    validate_text(job_text, "Job Description")
    # ----------------------------------------
    # Load Skills Database
    # ----------------------------------------
    validate_file(skills_file)
    skills = load_skills(skills_file)
    print("Resume PDF loaded successfully!")
    print("Job description loaded successfully!")
    print("Skills database loaded successfully!")
    # ----------------------------------------
    # Find Skills
    # ----------------------------------------
    resume_skills = find_skills(
        resume_text,
        skills
    )
    job_skills = find_skills(
        job_text,
        skills
    )
    # ----------------------------------------
    # Skill Matching
    # ----------------------------------------
    matched_skills, missing_skills, match_percentage = (
        calculate_skill_match(
            job_skills,
            resume_skills
        )
    )
    # ----------------------------------------
    # Text Similarity
    # ----------------------------------------
    text_similarity = calculate_text_similarity(
        resume_text,
        job_text
    )
    # ----------------------------------------
    # Resume Section Analysis
    # ----------------------------------------
    sections = detect_resume_sections(
        resume_text
    )
    # ----------------------------------------
    # Education Requirement
    # ----------------------------------------
    (
        education_matches,
        education_requirements,
        education_percentage
    ) = analyze_education_requirement(
        resume_text,
        job_text
    )
    # ----------------------------------------
    # Experience Requirement
    # ----------------------------------------
    (
        experience_matches,
        experience_requirements,
        experience_percentage
    ) = analyze_experience_requirement(
        resume_text,
        job_text
    )
    # ----------------------------------------
    # Calculate Overall Match Score
    # ----------------------------------------
    overall_score = (
        (match_percentage * 0.50)
        + (text_similarity * 0.30)
        + (experience_percentage * 0.20)
    )
    # ----------------------------------------
    # Resume Skills
    # ----------------------------------------
    print()
    print("---------- RESUME SKILLS ----------")
    for skill in resume_skills:
        print("✓", skill)
    # ----------------------------------------
    # Job Required Skills
    # ----------------------------------------
    print()
    print("-------- JOB REQUIRED SKILLS --------")
    for skill in job_skills:
        print("•", skill)
    # ----------------------------------------
    # Matched Skills
    # ----------------------------------------
    print()
    print("---------- MATCHED SKILLS ----------")
    for skill in matched_skills:
        print("✓", skill)
    # ----------------------------------------
    # Missing Skills
    # ----------------------------------------
    print()
    print("---------- MISSING SKILLS ----------")
    for skill in missing_skills:
        print("X", skill)
```

```python
    # Matching Result
    # ----------------------------------------
    print()
    print("========================================")
    print("          MATCHING RESULT")
    print("========================================")
    print("Required Skills :", len(job_skills))
    print("Matched Skills  :", len(matched_skills))
    print("Missing Skills  :", len(missing_skills))
    print()
    print(
        "Skill Match:",
        round(match_percentage, 2),
        "%"
    )
    print(
        "Text Similarity:",
        round(text_similarity, 2),
        "%"
    )
    # ----------------------------------------
    # Resume Section Analysis
    # ----------------------------------------
    print()
    print("========================================")
    print("       RESUME SECTION ANALYSIS")
    print("========================================")
    for section, found in sections.items():
        if found:
            print("√", section, ": Detected")
        else:
            print("X", section, ": Not Detected")
    print("========================================")
```

```python
    # ----------------------------------------
    print()
    print("========================================")
    print("       JOB REQUIREMENT ANALYSIS")
    print("========================================")
    # Education
    print()
    print("Education Requirement:")
    if education_matches:
        print(
            "√ Education requirement appears to be matched"
        )
    else:
        print(
            "X Education requirement not detected in resume"
        )
    print()
    print("Required Education Keywords:")
    for keyword in education_requirements:
        print("•", keyword)
    print("========================================")
    # Experience
    print()
    print("Experience Requirement:")
    print("----------------------------------------")
    if experience_matches:
        print("√ Relevant experience detected")
        print()
        print("Matched Experience Keywords:")
        for keyword in experience_matches:
            print("√", keyword)
    else:
        print("X Relevant experience not detected")
```

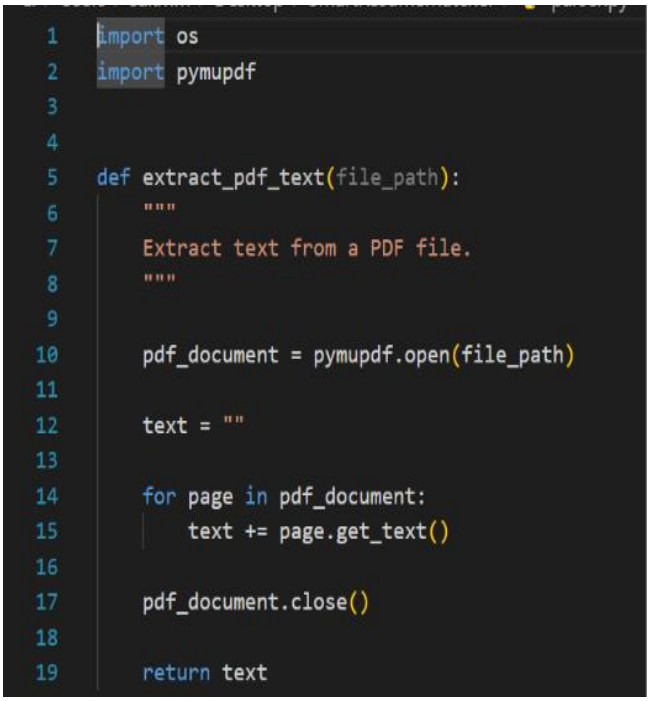
```python
    print()
    print(
        "Experience Keyword Match:",
        round(experience_percentage, 2),
        "%"
    )
    print("========================================")
    # ----------------------------------------
    # Final Match Analysis
    # ----------------------------------------
    print()
    print("========================================")
    print("          FINAL MATCH ANALYSIS")
    print("========================================")
    print(
        "Skill Match       :",
        round(match_percentage, 2),
        "%"
    )
    print(
        "Text Similarity   :",
        round(text_similarity, 2),
        "%"
    )
    print(
        "Experience Match  :",
        round(experience_percentage, 2),
        "%"
    )
    print()
    print(
        "Overall Match     :",
        round(overall_score, 2),
```

```python
    )
    print("========================================")
    # ----------------------------------------
    # Generate Final Report
    # ----------------------------------------
    report = generate_report(
        matched_skills,
        missing_skills,
        match_percentage,
        text_similarity,
        experience_percentage,
        overall_score
    )
    display_report(report)
# ----------------------------------------
# Error Handling
# ----------------------------------------
except FileNotFoundError as error:
    print()
    print("ERROR:", error)
except ValueError as error:
    print()
    print("ERROR:", error)
except Exception as error:
    print()
    print("UNEXPECTED ERROR:", error)
```

9.3 PDF Parser Module — parser.py

The parser.py module is responsible for extracting text from PDF resumes.

The project uses the **PyMuPDF** library to open the PDF document and extract text from each page.
The main function is:
extract_pdf_text(file_path)
The function receives the path of the resume PDF and returns the extracted text.
The extracted text is then passed to other modules for skill detection and analysis.



```python
import os
import pymupdf


def extract_pdf_text(file_path):
    """
    Extract text from a PDF file.
    """

    pdf_document = pymupdf.open(file_path)

    text = ""

    for page in pdf_document:
        text += page.get_text()

    pdf_document.close()

    return text
```

9.4 Skill Processing Module — skills.py

The skills.py module manages the project's technical skills database and performs skill detection.
It provides two primary functions:
load_skills(file_path)
find_skills(text, skills)

load_skills()

This function reads the skills database from:
data/skills.txt
Each non-empty line is treated as an individual skill.

find_skills()

This function searches the provided text for skills contained in the database.
Regular expressions are used with word-boundary checks to reduce incorrect matches.
For example, the implementation avoids treating a shorter skill such as C as a match inside another skill such as C++.

```python
import re
def load_skills(file_path):
    """
    Load skills from the skills database.
    """
    with open(file_path, "r") as file:
        skills = file.read().splitlines()
    return [skill.strip() for skill in skills if skill.strip()]
def find_skills(text, skills):
    """
    Find skills accurately in the given text.
    """
    found_skills = []
    text_lower = text.lower()
    for skill in skills:
        skill_pattern = (
            r"(?<!\w)"
            + re.escape(skill.lower())
            + r"(?!\w)"
        )
        if re.search(skill_pattern, text_lower):
            found_skills.append(skill)
    return found_skills
```

9.5 Matching Module — matcher.py

The matcher.py module performs the main comparison operations.

It contains two major functions:

calculate_skill_match()

This function compares the skills detected in the resume with the skills detected in the job description.

It produces:

- Matched skills
- Missing skills
- Skill matching percentage

The percentage is calculated using:

Skill Match Percentage =(Matched Skills / Required Skills) × 100

calculate_text_similarity()

This function compares the textual content of the resume and job description.

The implementation uses:

- TfidfVectorizer from Scikit-learn
- cosine_similarity from Scikit-learn

The resulting similarity value is converted into a percentage.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
def calculate_skill_match(job_skills, resume_skills):
    """
    Compare job-required skills with skills found in the resume.
    """
    matched_skills = [
        skill for skill in job_skills
        if skill in resume_skills
    ]
    missing_skills = [
        skill for skill in job_skills
        if skill not in resume_skills
    ]
    if len(job_skills) > 0:
        match_percentage = (
            len(matched_skills) / len(job_skills)
        ) * 100
    else:
        match_percentage = 0

    return matched_skills, missing_skills, match_percentage
def calculate_text_similarity(resume_text, job_text):
    """
    Calculate similarity between resume and job description
    using TF-IDF and cosine similarity.
    """
    documents = [resume_text, job_text]
    vectorizer = TfidfVectorizer()
    tfidf_matrix = vectorizer.fit_transform(documents)
    similarity = cosine_similarity(
        tfidf_matrix[0:1],
        tfidf_matrix[1:2]
    )[0][0] * 100
    return similarity
if __name__ == "__main__":
    resume_skills = ["Python", "SQL", "Flask", "Git"]
    job_skills = [
        "Python",
        "SQL",
        "Flask",
        "Git",
        "Docker",
        "Kubernetes",
        "AWS"
    ]
    matched, missing, score = calculate_skill_match(
        job_skills,
        resume_skills
    )
    print("Matched Skills:", matched)
    print("Missing Skills:", missing)
    print("Skill Match:", round(score, 2), "%")
    resume_text = """
    Python Flask SQL Git software development
    """
    job_text = """
    Python Flask SQL Git Docker Kubernetes AWS
    """
    similarity = calculate_text_similarity(
        resume_text,
        job_text
    )
    print("Text Similarity:", round(similarity, 2), "%")
```

9.6 Analysis Module — analyzer.py

The analyzer.py module performs additional analysis of the resume and job requirements.

It contains functions for:

- Detecting resume sections
- Analyzing education requirements
- Analyzing experience requirements

Resume Section Detection

The system checks whether the following sections are present:

- Skills
- Education
- Experience
- Projects

Education Analysis

The system searches for education-related terms such as:

- Bachelor
- B.Tech
- B.E.
- Degree
- Computer Science
- Computer Engineering

It compares relevant terms found in the job description with corresponding information in the resume.

Experience Analysis

The system searches for experience-related terms such as:

- Experience
- Python development
- Software development
- Web development
- Web applications
- Software developer
- Developer

The detected information is used to calculate an experience-related matching percentage.

```python
def detect_resume_sections(resume_text):
    """
    Detect important sections present in the resume.
    """
    sections = {
        "Skills": False,
        "Education": False,
        "Experience": False,
        "Projects": False
    }
    lines = resume_text.splitlines()
    for line in lines:
        line = line.strip().lower()
        if line == "skills:" or line == "skills":
            sections["Skills"] = True
        elif line == "education:" or line == "education":
            sections["Education"] = True
        elif line == "experience:" or line == "experience":
            sections["Experience"] = True
        elif line == "projects:" or line == "projects":
            sections["Projects"] = True
    return sections
def analyze_education_requirement(resume_text, job_text):
    """
    Analyze whether the resume contains education-related
    information required by the job description.
    """
    education_keywords = [
        "bachelor",
        "b.tech",
        "b.e.",
        "degree",
        "computer science",
        "computer engineering"
    ]
    job_lower = job_text.lower()
    resume_lower = resume_text.lower()
    required_keywords = []
    matched_keywords = []
    for keyword in education_keywords:
        if keyword in job_lower:
            required_keywords.append(keyword)
            if keyword in resume_lower:
                matched_keywords.append(keyword)
    if len(required_keywords) > 0:
        percentage = (
            len(matched_keywords) /
            len(required_keywords)
        ) * 100
    else:
        percentage = 100
    return matched_keywords, required_keywords, percentage
def analyze_experience_requirement(resume_text, job_text):
    """
    Analyze experience-related requirements in the job description.
    """
    experience_keywords = [
        "experience",
        "python development",
        "software development",
        "web development",
        "web applications",
        "software developer",
        "developer"
    ]
```

```
    job_lower = job_text.lower()
    resume_lower = resume_text.lower()
    required_keywords = []
    matched_keywords = []
    for keyword in experience_keywords:
        if keyword in job_lower:
            required_keywords.append(keyword)
            if keyword in resume_lower:
                matched_keywords.append(keyword)
    if len(required_keywords) > 0:
        percentage = (
            len(matched_keywords) /
            len(required_keywords)
        ) * 100
    else:
        percentage = 100
    return matched_keywords, required_keywords, percentage
if __name__ == "__main__":
    resume_text = """
    Computer Science Engineering Student
    Education:
    Bachelor of Technology in Computer Science Engineering
    Experience:
    Software Developer Intern with experience in Python and Flask.
    Projects:
    Developed web applications using Python and Flask.
    """
    job_text = """
    Software Developer
    Requirements:
    Bachelor degree in Computer Science.
    Experience with Python development and web applications.
    """
    sections = detect_resume_sections(resume_text)
    print("Resume Sections:")
    print(sections)
    matched_education, required_education, education_score = (
        analyze_education_requirement(
            resume_text,
            job_text
        )
    )
    print("\nEducation:")
    print("Matched:", matched_education)
    print("Required:", required_education)
    print("Match:", round(education_score, 2), "%")
    matched_experience, required_experience, experience_score = (
        analyze_experience_requirement(
            resume_text,
            job_text
        )
    )
    print("\nExperience:")
    print("Matched:", matched_experience)
    print("Required:", required_experience)
    print("Match:", round(experience_score, 2), "%")
```

9.7 Reporting Module — reporter.py

The reporter.py module generates and displays the final analysis report.

It provides functions for:

generate_recommendations()

generate_report()

display_report()

Recommendation Generation

The system generates recommendations based on skills missing from the candidate's resume.

For each missing skill, an improvement recommendation is generated.

Report Generation

The report combines:

- Matched skills
- Missing skills
- Skill match percentage
- Text similarity
- Experience match
- Overall match
- Recommendations

Report Display

The final report is displayed through the command-line interface in a structured format.

```python
def generate_recommendations(missing_skills):
    """
    Generate recommendations based on missing skills.
    """
    recommendations = []
    for skill in missing_skills:
        recommendations.append(
            f"Improve knowledge of {skill}"
        )
    return recommendations
def generate_report(
    matched_skills,
    missing_skills,
    skill_match_percentage,
    text_similarity,
    experience_percentage,
    overall_match
):
    """
    Generate the final resume-job matching report.
    """
    recommendations = generate_recommendations(missing_skills)
    report = {
        "strengths": matched_skills,
        "missing_skills": missing_skills,
        "recommendations": recommendations,
        "skill_match": skill_match_percentage,
        "text_similarity": text_similarity,
        "experience_match": experience_percentage,
        "overall_match": overall_match
    }
    return report
def display_report(report):
```

```python
def display_report(report):
    """
    Display the final analysis report in the terminal.
    """
    print()
    print("========================================")
    print("           MATCHING RESULTS")
    print("========================================")
    print()
    print(
        f"Skill Match: "
        f"{report['skill_match']:.2f}%"
    )
    print(
        f"Text Similarity: "
        f"{report['text_similarity']:.2f}%"
    )
    print(
        f"Experience Match: "
        f"{report['experience_match']:.1f}%"
    )
    print(
        f"Overall Match: "
        f"{report['overall_match']:.2f}%"
    )
    print()
    print("Strengths:")
    if report["strengths"]:
        for skill in report["strengths"]:
            print(f"✓ {skill}")
    else:
        print("None identified")
    print()
    print("Missing Skills:")
    if report["missing_skills"]:
        for skill in report["missing_skills"]:
            print(f"X {skill}")
    else:
        print("None")
    print()
    print("Recommendations:")
    if report["recommendations"]:
        for recommendation in report["recommendations"]:
            print(f"→ {recommendation}")
    else:
        print("No recommendations")
    print()
    print("========================================")
    print("          ANALYSIS COMPLETE")
    print("========================================")
if __name__ == "__main__":
    matched_skills = [
        "Python",
        "SQL",
        "Flask",
        "Git"
    ]
```

```python
    missing_skills = [
        "Docker",
        "Kubernetes",
        "AWS"
    ]
    report = generate_report(
        matched_skills,
        missing_skills,
        57.14,
        55.12,
        80.0,
        61.11
    )
    display_report(report)
```

9.8 Utility Module — utils.py

The utils.py module contains reusable validation and utility functions.

The module provides:

validate_file()

Checks whether a required file exists and is a valid file.

read_text_file()

Reads text files using UTF-8 encoding.

validate_text()

Checks whether input text is empty.

format_percentage()

Formats numerical values as percentage strings.

Centralizing these operations avoids unnecessary duplication across the project.

```python
import os
def validate_file(file_path):
    """
    Check whether a file exists and is accessible.
    """
    if not os.path.exists(file_path):
        raise FileNotFoundError(
            f"Required file not found: {file_path}"
        )
    if not os.path.isfile(file_path):
        raise ValueError(
            f"Path is not a valid file: {file_path}"
        )
    return True
def read_text_file(file_path):
    """
    Read and return the contents of a text file.
    """
    validate_file(file_path)
    with open(file_path, "r", encoding="utf-8") as file:
        return file.read()
def validate_text(text, field_name="Text"):
    """
    Validate that text input is not empty.
    """
    if not text or not text.strip():
        raise ValueError(
            f"{field_name} cannot be empty."
        )
    return True
def format_percentage(value, decimal_places=2):
    """
    Format a numeric value as a percentage string.
    """
    return f"{value:.{decimal_places}f}%"
if __name__ == "__main__":
    test_file = "job.txt"
    try:
        validate_file(test_file)
        content = read_text_file(test_file)
        validate_text(content, "Job Description")
        print("Utils module test successful!")
        print()
        print("Job description loaded successfully.")
        print()
        print("Formatted percentage:", format_percentage(61.1111))
    except (FileNotFoundError, ValueError) as error:
        print("Error:", error)
```

9.9 Input Data

The project uses three primary input sources.

Resume

samples/resume.pdf

Contains the candidate's resume information.

Tanisi Agarwal

Computer Science Engineering Student

Skills:

Python,Java,C++,SQL

Education:

Bachelor of Technology in Computer Science Engineering

Experience:

Software Developer Intern with experience in Python and Flask.

Projects:

Developed web applications using Python and Flask.

Worked with SQL databases and Git.

Job Description

samples/job.txt

Contains the requirements for the target position.

```
Software Developer

We are looking for a Software Developer with experience in Python,
Flask, SQL, Git and Docker.

Requirements:

Bachelor's degree in Computer Science or related field.

Experience with Python development and web applications.

Knowledge of AWS and Kubernetes is preferred.
```

Skills Database

data/skills.txt

Contains the list of technical skills used during skill detection.

The skills database can be modified to support additional skills.



9.10 Output Generation

After processing all inputs, the application produces a command-line report containing:

- Required skills
- Resume skills
- Matched skills
- Missing skills
- Skill match percentage
- Text similarity percentage
- Resume section analysis
- Education requirement analysis
- Experience requirement analysis
- Overall match percentage
- Recommendations

This provides the user with a consolidated view of the comparison results.

```
========================================
     SMART RESUME & JOB MATCHER
========================================

Resume PDF loaded successfully!
Job description loaded successfully!
Skills database loaded successfully!

---------- RESUME SKILLS ----------
✓ Python
✓ Java
✓ C++
✓ SQL
✓ Flask
✓ Git

-------- JOB REQUIRED SKILLS --------
• Python
• SQL
• Flask
• Git
• Docker
• Kubernetes
• AWS

---------- MATCHED SKILLS ----------
✓ Python
✓ SQL
✓ Flask
✓ Git

---------- MISSING SKILLS ----------
✗ Docker
✗ Kubernetes
✗ AWS
```

```
========================================
          MATCHING RESULT
========================================
Required Skills : 7
Matched Skills  : 4
Missing Skills  : 3

Skill Match: 57.14 %
Text Similarity: 55.12 %

========================================
       RESUME SECTION ANALYSIS
========================================
✓ Skills : Detected
✓ Education : Detected
✓ Experience : Detected
✓ Projects : Detected
========================================

========================================
       JOB REQUIREMENT ANALYSIS
========================================

Education Requirement:
✓ Education requirement appears to be matched

Required Education Keywords:
• bachelor
• degree
• computer science
========================================

Experience Requirement:
----------------------------------------
```

```
Experience Requirement:
----------------------------------------
✓ Relevant experience detected

Matched Experience Keywords:
✓ experience
✓ web applications
✓ software developer
✓ developer

Experience Keyword Match: 80.0 %
========================================

========================================
          FINAL MATCH ANALYSIS
========================================
Skill Match        : 57.14 %
Text Similarity    : 55.12 %
Experience Match   : 80.0 %

Overall Match      : 61.11 %
========================================

========================================
           MATCHING RESULTS
========================================

Skill Match: 57.14%
Text Similarity: 55.12%
Experience Match: 80.0%
Overall Match: 61.11%
```

```
Strengths:
✓ Python
✓ SQL
✓ Flask
✓ Git

Missing Skills:
✗ Docker
✗ Kubernetes
✗ AWS

Recommendations:
→ Improve knowledge of Docker
→ Improve knowledge of Kubernetes
→ Improve knowledge of AWS

========================================
          ANALYSIS COMPLETE
========================================
```

9.11 Overall Score Calculation

The project combines multiple analysis components into an overall matching score.

The current implementation uses the following formula:

Overall Match = (Skill Match × 0.50) + (Text Similarity × 0.30) + (Experience Match × 0.20)

The skill match component receives the largest weight because technical skill overlap is a major part of the implemented comparison.

Text similarity provides an additional measure of overall textual alignment, while experience matching contributes to the final score.

These weights are project-specific design decisions and are not intended to represent an industry-standard recruitment formula.

10. Testing and Results

10.1 Testing Overview

Testing was performed to verify that the major components of the Smart Resume & Job Matching System work correctly and produce the expected results.

The project includes individual unit-test files for important modules. Testing was focused on the core functionality of PDF parsing, skill detection, skill matching, and resume analysis.

The testing files are located in:

```
tests/
├── test_parser.py
├── test_skills.py
├── test_matcher.py
└── test_analyzer.py
```

10.2 Unit Testing

Unit testing was used to test individual modules independently.

Parser Testing

The parser module was tested to verify that text could be successfully extracted from the provided PDF resume.

Tested functionality:

- PDF file loading
- Text extraction
- Returned text validation

The parser test completed successfully.

```python
import os
import sys
import unittest

# Add the project root to Python's import path
PROJECT_ROOT = os.path.abspath(
    os.path.join(os.path.dirname(__file__), "..")
)

sys.path.insert(0, PROJECT_ROOT)

from parser import extract_pdf_text


class TestParser(unittest.TestCase):

    def test_pdf_extraction(self):
        resume_path = os.path.join(
            PROJECT_ROOT,
            "resume.pdf"
        )

        text = extract_pdf_text(resume_path)

        self.assertIsInstance(text, str)
        self.assertGreater(len(text.strip()), 0)


if __name__ == "__main__":
    unittest.main()
```

Skill Detection Testing

The skill detection module was tested to verify that skills could be correctly loaded from the skills database and detected in input text.

Tested functionality:

- Loading skills
- Searching for skills
- Correct skill identification
- Word-boundary-based matching

The skill detection test completed successfully.

```python
import os
import sys
import unittest
PROJECT_ROOT = os.path.abspath(
    os.path.join(os.path.dirname(__file__), "..")
)
sys.path.insert(0, PROJECT_ROOT)
from skills import load_skills, find_skills
class TestSkills(unittest.TestCase):
    def test_load_skills(self):
        skills_file = os.path.join(
            PROJECT_ROOT,
            "skills.txt"
        )
        skills = load_skills(skills_file)
        self.assertIsInstance(skills, list)
        self.assertGreater(len(skills), 0)
    def test_find_skills(self):
        skills = [
            "Python",
            "Java",
            "C++",
            "SQL",
            "Flask"
        ]
        text = "I have experience with Python, C++, SQL and Flask."
        found_skills = find_skills(
            text,
            skills
        )
        self.assertIn("Python", found_skills)
        self.assertIn("C++", found_skills)
        self.assertIn("SQL", found_skills)
        self.assertIn("Flask", found_skills)
    def test_skill_not_found(self):
        skills = [
            "Python",
            "Java",
            "SQL"
        ]
        text = "I have experience with Python."
        found_skills = find_skills(
            text,
            skills
        )
        self.assertNotIn("Java", found_skills)
        self.assertNotIn("SQL", found_skills)
if __name__ == "__main__":
    unittest.main()
```

Matching Testing

The matching module was tested to verify the calculation of matched skills, missing skills, and text similarity.

Tested functionality:

- Skill comparison
- Matched skill identification
- Missing skill identification
- Skill match percentage
- TF-IDF text similarity

The matching test completed successfully.

```python
import os
import sys
import unittest
PROJECT_ROOT = os.path.abspath(
    os.path.join(os.path.dirname(__file__), "..")
)
sys.path.insert(0, PROJECT_ROOT)
from matcher import (
    calculate_skill_match,
    calculate_text_similarity
)
class TestMatcher(unittest.TestCase):
    def test_skill_match(self):
        resume_skills = [
            "Python",
            "SQL",
            "Flask",
            "Git"
        ]
        job_skills = [
            "Python",
            "SQL",
            "Flask",
            "Git",
            "Docker",
            "AWS"
        ]
        matched, missing, percentage = calculate_skill_match(
            job_skills,
            resume_skills
        )
        self.assertEqual(
            matched,
            ["Python", "SQL", "Flask", "Git"]
        )
        self.assertEqual(
            missing,
            ["Docker", "AWS"]
        )
        self.assertAlmostEqual(
            percentage,
            66.6667,
            places=2
        )
    def test_empty_job_skills(self):
        matched, missing, percentage = calculate_skill_match(
            [],
            ["Python"]
        )
        self.assertEqual(matched, [])
        self.assertEqual(missing, [])
        self.assertEqual(percentage, 0)
    def test_text_similarity(self):
        resume_text = """
        Python Flask SQL software development
        """
        job_text = """
        Python Flask SQL software development
        """
        similarity = calculate_text_similarity(
            resume_text,
            job_text
        )
        self.assertAlmostEqual(
            similarity,
            100.0,
            places=2
        )
if __name__ == "__main__":
    unittest.main()
```

Analyzer Testing

The analyzer module was tested to verify resume section detection and requirement analysis.

Tested functionality:

- Resume section detection
- Education requirement analysis
- Experience requirement analysis

The analyzer test completed successfully

```python
import os
import sys
import unittest
PROJECT_ROOT = os.path.abspath(
    os.path.join(os.path.dirname(__file__), "..")
)
sys.path.insert(0, PROJECT_ROOT)
from analyzer import (
    detect_resume_sections,
    analyze_education_requirement,
    analyze_experience_requirement
)
class TestAnalyzer(unittest.TestCase):
    def setUp(self):
        self.resume_text = """
        Computer Science Engineering Student
        Skills:
        Python, Java, SQL
        Education:
        Bachelor of Technology in Computer Science Engineering
        Experience:
        Software Developer Intern with experience in Python and Flask.
        Projects:
        Developed web applications using Python and Flask.
        """
        self.job_text = """
        Software Developer
        Requirements:
        Bachelor degree in Computer Science.
        Experience with Python development and web applications.
        """
    def test_resume_sections(self):
        sections = detect_resume_sections(
            self.resume_text
        )
        self.assertTrue(sections["Skills"])
        self.assertTrue(sections["Education"])
        self.assertTrue(sections["Experience"])
        self.assertTrue(sections["Projects"])
    def test_education_analysis(self):
        matched, required, percentage = (
            analyze_education_requirement(
                self.resume_text,
                self.job_text
            )
        )
        self.assertGreater(len(required), 0)
        self.assertGreater(len(matched), 0)
        self.assertGreater(percentage, 0)
    def test_experience_analysis(self):
        matched, required, percentage = (
            analyze_experience_requirement(
                self.resume_text,
                self.job_text
            )
        )
        self.assertGreater(len(required), 0)
        self.assertGreater(len(matched), 0)
        self.assertGreater(percentage, 0)
if __name__ == "__main__":
    unittest.main()
```

10.3 Error Handling Testing

The application was also tested for missing input files.

During testing, the resume PDF was temporarily unavailable and the application was executed.

The application correctly detected that the required file was missing and displayed an appropriate error message instead of continuing with invalid input.

After testing, the input file was restored to its original location.

This verifies that the application's file validation and exception-handling mechanism is functioning as intended.

10.4 Functional Test Cases

Test Case	Input/Condition	Expected Result	Status
TC-01	Valid resume PDF	Resume text is extracted	Passed
TC-02	Valid job description	Job description is loaded	Passed
TC-03	Valid skills database	Skills are loaded successfully	Passed
TC-04	Resume containing known skills	Skills are detected	Passed
TC-05	Resume and job with overlapping skills	Matched skills are identified	Passed
TC-06	Job contains skills absent from resume	Missing skills are identified	Passed
TC-07	Valid resume and job text	Text similarity is calculated	Passed
TC-08	Resume containing recognized sections	Sections are detected	Passed
TC-09	Job containing education requirements	Education requirements are analyzed	Passed
TC-10	Job containing experience requirements	Experience requirements are analyzed	Passed
TC-11	Missing resume file	Appropriate error is displayed	Passed
TC-12	Complete valid input	Final matching report is generated	Passed

10.5 Sample Execution Results

The system was executed using the provided sample resume and job description.

The analysis produced the following results:

Metric	Result
Skill Match	57.14%
Text Similarity	55.12%
Experience Match	80.00%
Overall Match	61.11%

The system also identified matched and missing skills and generated recommendations based on the missing skills.

These values represent the output of the implemented algorithms for the particular sample resume and job description. They should not be interpreted as an objective employment suitability score.

10.6 Result Analysis

The generated results demonstrate that the system can perform multiple forms of resume-job comparison.

The **Skill Match** value represents the proportion of job-required skills detected in the resume.

The **Text Similarity** value represents the similarity between the resume and job description based on TF-IDF representations and cosine similarity.

The **Experience Match** value represents the proportion of detected experience-related requirements that were also identified in the resume.

The **Overall Match** combines the implemented scoring components using the project's predefined weights.

The system also provides qualitative information through matched skills, missing skills, and recommendations, allowing users to understand the reasons behind the calculated results rather than relying only on a single percentage.

---

11. Challenges Faced

The main challenges faced during development were:

- Extracting readable text from PDF resumes.
- Detecting skills accurately using text matching.
- Avoiding incorrect matches such as C inside C++.
- Combining skill, text, and experience analysis into one score.
- Handling missing or invalid input files.
- Organizing the project into separate Python modules.

---

12. Learning Outcomes

This project helped develop practical knowledge of:

- Python programming and control flow.
- Functions and modular programming.
- Lists, dictionaries, and string processing.
- File handling and exception handling.
- Regular expressions.
- Python libraries such as PyMuPDF and Scikit-learn.
- TF-IDF and cosine similarity.
- Unit testing and software documentation.

---

13. Limitations

- The system currently works through a command-line interface.
- Skill detection depends on the predefined skills database.
- The system primarily uses keyword-based requirement analysis.
- The matching score is project-specific and is not an actual hiring decision.
- The current version does not include a database or user authentication.

---

14. Future Enhancements

Future versions could include:

- Web-based user interface.
- Database integration.

- User authentication.
- Support for multiple resume formats.
- Larger and more dynamic skill databases.
- Improved semantic text matching.
- Resume improvement suggestions.
- Job recommendation functionality.

---

15. Conclusion

The Smart Resume & Job Matching System successfully demonstrates how Python can be used to automate resume and job-description analysis. The system extracts resume information, detects skills, compares job requirements, calculates text similarity, analyzes education and experience, and generates recommendations.

The project applies Python programming concepts in a practical real-world application and demonstrates modular design, file handling, text processing, error handling, and testing.

---

16. References

1. VITyarthi — Python Essentials Course
2. Python Documentation : https://docs.python.org/3/library/re.html?utm_source=chatgpt.com
3. PyMuPDF Documentation : https://pymupdf.readthedocs.io/en/latest/?utm_source=chatgpt.com
4. Scikit-learn Documentation : https://scikit-learn.org/stable/modules/metrics.html?utm_source=chatgpt.com
5. TF-IDF and Cosine Similarity concepts